Tổng quan kiến trúc RAG

Nguyễn Lê Anh Tuấn*, Nguyễn Đức Huy†, Lê Thành Nam‡, Hà Minh Châu§
HYCSC / Khoa CNTT / Trường Đại học Xây Dựng Hà Nội
Email: *tuan.nlaforwork@gmail.com, †huy.nd4work@gmail.com, ‡nam2904thanh@gmail.com, §email4@domain.com

***Tóm tắt nội dung*—Mặc dù các mô hình ngôn ngữ lớn (LLMs) đạt được nhiều tiến bộ, chúng vẫn gặp hạn chế về hiện tượng ảo giác, tri thức khó cập nhật sau huấn luyện và khả năng kiểm chứng nguồn của câu trả lời. Kiến trúc tạo sinh tăng cường truy xuất (RAG) kết hợp bộ nhớ tham số của mô hình ngôn ngữ với nguồn tri thức bên ngoài được truy xuất khi cần. Tài liệu tổng hợp này hệ thống hóa sự tiến hóa của công nghệ RAG qua ba thế hệ: *Naive RAG* với quy trình tuyến tính cơ bản; *Advanced RAG* tập trung tối ưu hóa tiền và hậu truy xuất; cùng *Modular RAG* mang lại kiến trúc mô-đun hóa linh hoạt. Bên cạnh đó, tài liệu đi sâu phân tích *Self-RAG*—một bước ngoặt kỹ thuật cho phép mô hình thực hiện truy xuất thích ứng theo nhu cầu và tự phản biện chất lượng đầu ra thông qua hệ thống token phản biện. Bài viết cũng phân tích cơ chế xác suất của RAG-Sequence và RAG-Token, quá trình huấn luyện và giải mã, cùng khả năng truy xuất thích ứng và tự đánh giá của Self-RAG.**

***Index Terms*—Tạo sinh Tăng cường Truy xuất (RAG), Mô hình Ngôn ngữ Lớn (LLMs), Bộ nhớ tham số, Bộ nhớ phi tham số, Self-RAG, Token phản biện, Hiện tượng ảo giác.**

I. Giới thiệu chung và khái niệm cốt lõi của RAG

A. Bối cảnh ra đời và Hạn chế của LLM

1) Bối cảnh ra đời và Khái niệm Bộ nhớ tham số (Parametric Memory): Các Mô hình Ngôn ngữ Lớn (LLMs) dựa trên kiến trúc Transformer tiền huấn luyện (như BART, T5, GPT, LLaMA) đã tạo ra bước ngoặt lớn trong xử lý ngôn ngữ tự nhiên [1], [2]. Trong quá trình tiền huấn luyện trên quy mô dữ liệu khổng lồ, mô hình tự động học và lưu trữ một lượng tri thức thực tế trực tiếp vào hệ thống các trọng số (parameters) [1]. Tập hợp tri thức được mã hóa ngầm định này được gọi là Bộ nhớ tham số (*Parametric Memory*) hay cơ sở tri thức ngầm định hóa [1], [2]. Nhờ bộ nhớ tham số, LLM có khả năng hiểu ngữ cảnh, suy luận và tạo sinh văn bản vô cùng trôi chảy mà không cần truy cập vào bất kỳ bộ nhớ ngoài nào trong quá trình suy luận [1].

2) Các Hạn chế Cốt lõi của LLM Thuần (Parametric-only LLMs): Mặc dù sở hữu năng lực tạo sinh ấn tượng, các mô hình chỉ dựa vào bộ nhớ tham số phải đối mặt với nhiều rào cản kỹ thuật nghiêm trọng [1], [2]:

- **Tri thức bị đóng băng và nhanh lạc hậu (*Outdated Knowledge*):** Tri thức được mã hóa trong tham số phụ thuộc vào dữ liệu huấn luyện, khiến mô hình khó đáp ứng chính xác các câu hỏi về những sự kiện mới nếu không có nguồn bổ sung [1], [2].
- **Khó cập nhật và chỉnh sửa tri thức (*Rigid Memory*):** Việc bổ sung kiến thức mới hoặc sửa đổi các thông tin sai lệch đã lưu trong trọng số đòi hỏi phải huấn luyện lại (*re-train*) hoặc tinh chỉnh (*fine-tuning*) đắt đỏ, tiêu tốn rất nhiều tài nguyên tính toán [1], [2].
- **Thiếu tính minh bạch và không thể truy xuất nguồn gốc (*Lack of Provenance/Traceability*):** LLM thuần tham số không tự bảo đảm rằng các trích dẫn được tạo ra có thể kiểm chứng hay phản ánh đúng nguồn hình thành câu trả lời [1]–[3].
- **Hạn chế trong các tác vụ tri thức chuyên sâu và ít phổ biến (*Knowledge-Intensive & Long-Tail Knowledge*):** Đối với các câu hỏi thuộc miền chuyên ngành hẹp hoặc thông tin hiếm gặp (*long-tail knowledge*), năng lực suy luận của LLM dựa trên bộ nhớ tham số bị giảm sút rõ rệt [1], [2].

3) Hiện tượng "Ảo giác" (Hallucination) và Nguyên nhân Sâu xa: Ảo giác là hiện tượng LLM tạo ra các câu trả lời nghe rất thuyết phục, đúng cấu trúc ngữ pháp và tự tin, nhưng nội dung bên trong lại sai lệch sự thật (*factually incorrect*) hoặc hoàn toàn không có căn cứ, không được hỗ trợ bởi thực tế hay tài liệu tham chiếu [2], [3].

Nguyên nhân sâu xa khiến LLM gặp hiện tượng ảo giác bao gồm:

1) **Bản chất sinh từ theo xác suất (*Next-token Prediction*):** LLM hoạt động dựa trên việc tính toán xác suất để dự đoán từ tiếp theo dựa vào các chuỗi đã học trong bộ nhớ tham số, chứ không đối chiếu với thực tế hay kho tài liệu chuẩn mực khi suy luận [2], [3].
2) **Áp lực trả lời các truy vấn vượt quá tập dữ liệu:** Khi gặp câu hỏi nằm ngoài tập dữ liệu huấn luyện hoặc thông tin quá mới, thay vì thừa nhận "không biết", mô hình vẫn cố gắng ghép nối các từ ngữ có xác suất cao nhất để tạo thành câu hoàn chỉnh [2].
3) **Giới hạn của mục tiêu dự đoán token:** Mục tiêu dự đoán token không trực tiếp bảo đảm tính đúng đắn của các phát biểu. Vì vậy, câu trả lời có xác suất cao và diễn đạt trôi chảy vẫn có thể thiếu căn cứ thực tế [2], [3].

B. Kiến trúc Bộ nhớ Kép (Hybrid Memory Architecture)

Để giải quyết các hạn chế vốn có của các mô hình ngôn ngữ lớn thuần tham số, kiến trúc Tạo sinh Tăng cường Truy xuất (RAG) kết hợp hài hòa giữa Bộ nhớ tham số (*Parametric Memory*) và Bộ nhớ phi tham số (*Non-parametric Memory*) [1], [2].

1) Bộ nhớ tham số (Parametric Memory):

- **Bản chất:** Bộ nhớ tham số bao gồm toàn bộ hệ thống các trọng số (parameters θ) của một mô hình học sâu sinh chuỗi (*sequence-to-sequence / seq2seq*) đã được tiền huấn luyện, điển hình như BART-large [1].
- **Vai trò:** Bộ nhớ này hoạt động như một cơ sở tri thức ngầm định, chịu trách nhiệm học cú pháp ngôn ngữ, ngữ

pháp, khả năng tổng hợp thông tin và suy luận văn bản [1], [2].

- **Đặc điểm:** Tri thức tham số được cố định sau giai đoạn tiền huấn luyện; mô hình khai thác bộ nhớ này để đọc hiểu câu hỏi và sinh ra câu trả lời tự nhiên, mượt mà dựa trên ngữ cảnh được cung cấp [1], [2].

2) Bộ nhớ phi tham số (Non-parametric Memory):

- **Bản chất:** Bộ nhớ phi tham số là một kho tri thức độc lập bên ngoài mô hình ngôn ngữ, lưu trữ văn bản thô dưới dạng chỉ mục vector mật (*Dense Vector Index*) [1].
- **Cấu trúc thực thi:** Trong thiết kế ban đầu của Lewis *et al.*, nguồn tri thức phi tham số được xây dựng từ tập dữ liệu Wikipedia (*Wikipedia dump*), chia nhỏ thành các đoạn văn bản cố định (*chunks*) gồm 100 từ [1].
- **Cơ chế biểu diễn & truy xuất:** Mỗi đoạn văn bản được mã hóa thành một vector nhúng mật thông qua bộ mã hóa tài liệu BERT thuộc mô hình Dense Passage Retriever (DPR) [1]. Các vector này được định vị trong không gian nhiều chiều và truy xuất với độ trễ thấp thông qua thuật toán Tìm kiếm Tích trong Tối đại (*Maximum Inner Product Search*—MIPS) [1].

3) Sự Tương tác và Cơ chế Cập nhật Tức thì (Index Hot-Swapping):

- **Quy trình phối hợp:** Khi nhận truy vấn từ người dùng, bộ truy xuất (*Retriever*) áp dụng thuật toán MIPS để lọc ra các đoạn văn bản liên quan nhất từ bộ nhớ phi tham số [1]. Các đoạn này được ghép nối trực tiếp với truy vấn gốc làm ngữ cảnh đầu vào cho mô hình BART sinh chuỗi phản hồi cuối cùng [1].
- **Khả năng cập nhật nguồn tri thức (*Index Hot-Swapping*):** Điểm ưu việt của kiến trúc kết hợp là khả năng cập nhật kho tri thức mà không cần tái huấn luyện (*re-train*) các trọng số θ tốn kém tài nguyên [1], [2]. Khi tri thức thực tế biến động hoặc hệ thống mở rộng sang miền nghiệp vụ mới, người vận hành chỉ cần thay thế hoặc nạp lại tập chỉ mục vector ngay tại thời điểm thực thi (*test-time*) [1].

C. Lợi ích Vượt trội của RAG

So với các mô hình ngôn ngữ lớn chỉ dựa vào bộ nhớ tham số thuần túy (*parametric-only LLMs*), kiến trúc RAG mang lại nhiều lợi thế kỹ thuật đóng vai trò quyết định trong việc triển khai trí tuệ nhân tạo vào thực tế [1], [2].

1) Giảm thiểu Hiện tượng "Ảo giác" và Tăng Độ chính xác Thực tế:

- **Cung cấp căn cứ thực tế:** Việc truy xuất các đoạn văn bản có liên quan từ cơ sở dữ liệu bên ngoài đóng vai trò làm ngữ cảnh tham chiếu trực tiếp, nhờ đó có thể hạn chế việc mô hình tự tạo sinh các thông tin sai lệch thực tế [1]–[3].
- **Tối ưu cho các tác vụ tri thức chuyên sâu (*Knowledge-Intensive Tasks*):** RAG đặc biệt vượt trội khi xử lý các truy vấn đòi hỏi độ chính xác cao, tri thức chuyên ngành hẹp hoặc thông tin ít phổ biến (*long-tail knowledge*) mà bộ nhớ tham số của LLM dễ bỏ sót [1], [2].

2) Cập nhật Tri thức Linh hoạt và Tiết kiệm Chi phí (Index Hot-Swapping):

- **Cập nhật tri thức từ nguồn bên ngoài:** Cơ sở tri thức có thể được bổ sung hoặc thay thế thông qua việc cập nhật tài liệu và chỉ mục truy xuất trong bộ nhớ phi tham số thông qua kỹ thuật *Index Hot-Swapping* [1], [2].
- **Tiết kiệm tài nguyên tính toán:** Khả năng cập nhật tri thức bên ngoài giúp hệ thống tránh được quá trình tái huấn luyện (*re-train*) hoặc tinh chỉnh (*fine-tune*) tốn kém trên hàng tỷ trọng số của LLM [1], [2].

3) Tính Minh bạch và Khả năng Truy xuất Nguồn gốc (Traceability & Provenance):

- **Liên kết thông tin với nguồn tham chiếu:** RAG có thể lưu và hiển thị nguồn tài liệu tham chiếu (tên tài liệu, đoạn văn bản hoặc trang web) để người dùng đối chiếu với câu trả lời [1]–[3].
- **Cải thiện khả năng kiểm chứng nguồn:** Người dùng và chuyên gia có thể trực tiếp tra cứu, đối chiếu căn cứ của câu trả lời từ tài liệu gốc, giúp quy trình suy luận trở nên minh bạch và đáng tin cậy hơn so với mô hình thuần tham số [1], [2].

4) Tính Linh hoạt và Khả năng Kiểm soát Cao:

- **Mở rộng nguồn dữ liệu đa dạng:** RAG không giới hạn ở văn bản thô mà có thể tích hợp linh hoạt với dữ liệu bán cấu trúc, đồ thị tri thức (*Knowledge Graphs*) cũng như mở rộng sang dữ liệu đa phương thức (*multimodal*) [2].
- **Khả năng tự điều chỉnh hành vi khi suy luận:** Các kiến trúc hiện đại như Self-RAG cho phép mô hình thực hiện truy xuất thích ứng theo nhu cầu (*Adaptive Retrieval*) và tùy biến trọng số đánh giá ngay trong thời gian suy luận (*inference-time customization*) mà không cần huấn luyện lại [2], [3].

II. Mô hình RAG nguyên bản và cơ chế nền tảng

Trong mục này, x ký hiệu truy vấn, z là đoạn tài liệu truy xuất, $y = (y_1, \ldots, y_N)$ là chuỗi gồm N token, η và θ lần lượt là tham số của Retriever và Generator BART. Tập $\mathcal{Z}_K(x)$ gồm K đoạn được truy xuất cho x. Ký hiệu $p_\eta(z \mid x)$ biểu diễn trọng số truy xuất được sử dụng khi kết hợp các tài liệu; trong các ví dụ số, trọng số này được giả định chuẩn hóa trên tập top-K.

A. Mô hình Xác suất Điều kiện và Công thức Cấu thành RAG

Trong thiết kế của Lewis *et al.*, RAG được mô hình hóa toán học như một bài toán tối ưu hóa xác suất điều kiện với biến ngầm định (*latent variable*) là đoạn văn bản truy xuất z [1]. Để giảm chi phí trên không gian tài liệu lớn, RAG xấp xỉ phép biên duyên hóa (*marginalization*) trên tập *top-K* đoạn văn bản có xác suất truy xuất cao nhất [1]. Tùy thuộc vào mức độ phân giải của quá trình sinh văn bản, RAG được phân tách thành hai biến thể mô hình chính [1]:

a) Mô hình RAG-Sequence (Biên duyên hóa cấp chuỗi): Mô hình RAG-Sequence giả định rằng toàn bộ chuỗi câu trả lời y được điều kiện hóa trên cùng một tài liệu z duy nhất [1]. Xác suất tổng thể của câu trả lời y được biểu diễn bằng tổng xác suất sinh cả câu trên từng tài liệu, có trọng số là xác suất truy xuất của tài liệu đó [1]:

$$p_{\text{Seq}}(y \mid x) \approx \sum_{z \in \mathcal{Z}_K(x)} p_\eta(z \mid x) \cdot \prod_{i=1}^{N} p_\theta(y_i \mid x, z, y_{<i}). \tag{1}$$

b) Mô hình RAG-Token (Biên duyên hóa cấp token): Ngược lại, mô hình RAG-Token cho phép các đoạn văn bản khác nhau đóng góp vào việc sinh các token khác nhau trong cùng một câu trả lời [1]. Mô hình tính toán phân bố xác suất cho từng token tiếp theo bằng cách kết hợp thông tin từ nhiều tài liệu trước khi bước sang vị trí từ kế tiếp [1]:

$$p_{\text{Tok}}(y \mid x) \approx \prod_{i=1}^{N} \left(\sum_{z \in \mathcal{Z}_K(x)} p_\eta(z \mid x) \cdot p_\theta(y_i \mid x, z, y_{<i}) \right). \tag{2}$$

Ý nghĩa chi tiết của các thành phần toán học trong công thức [1]:

- **Biến đầu vào và đầu ra:**
  - x (*Input Query*): Câu hỏi, yêu cầu hoặc truy vấn đầu vào từ phía người dùng [1].
  - z (*Retrieved Passage*): Một đoạn văn bản ngữ cảnh được truy xuất từ cơ sở dữ liệu vector bên ngoài thông qua bộ truy xuất [1].
  - $y = (y_1, y_2, \ldots, y_N)$ (*Target Output*): Chuỗi câu trả lời hoàn chỉnh gồm N token mục tiêu được sinh ra tuần tự [1].
- **Thành phần truy xuất —** $p_\eta(z \mid x)$ **(Xác suất truy xuất / Retriever Score):** Biểu thị mức độ tương đồng ngữ nghĩa giữa câu hỏi x và tài liệu z [1]. Giá trị này do mô hình DPR tính toán bằng tích vô hướng giữa hai vector nhúng mật và chuẩn hóa qua hàm Softmax: $p_\eta(z \mid x) \propto \exp(\mathbf{d}(z)^T \mathbf{q}(x))$ [1].
- **Thành phần sinh chuỗi —** $\prod_{i=1}^{N} p_\theta(y_i \mid x, z, y_{<i})$ **(Xác suất sinh trọn câu trên 1 tài liệu):** Đại diện cho xác suất mô hình sinh chuỗi (BART) tạo ra trọn vẹn toàn bộ chuỗi y khi chỉ dựa vào một ngữ cảnh tài liệu z cố định [1]. Ký hiệu tích chuỗi $\prod$ thể hiện quy tắc nhân xác suất có điều kiện: xác suất của toàn câu bằng tích xác suất của từng token liên tiếp [1].
- **Thành phần sinh token —** $p_\theta(y_i \mid x, z, y_{<i})$ **(Xác suất sinh token tiếp theo):** Xác suất điều kiện để mô hình ngôn ngữ dự đoán ra token thứ i tiếp theo, dựa trên sự kết hợp giữa truy vấn gốc x, tài liệu tham chiếu z và chuỗi tiền tố $y_{<i}$ [1].
- **Phép toán tổng** $\sum_{z \in \mathcal{Z}_K(x)}$ **(Tổng có trọng số / Marginalization):** Phép tính biên duyên hóa xấp xỉ bằng cách lấy dự đoán trên từng tài liệu nhân với trọng số truy xuất tương ứng $p_\eta(z \mid x)$, sau đó tính tổng trên toàn bộ K đoạn văn bản có độ tương quan cao nhất để đưa ra quyết định dự đoán cuối cùng [1].

Các phân bố xác suất trong hai công thức không trực tiếp tạo thành câu trả lời cuối cùng. Chúng được sử dụng để xác định hàm mất mát trong huấn luyện và đánh giá các chuỗi ứng viên trong quá trình giải mã. Hai biến thể khác nhau chủ yếu ở cấp độ mà chúng thực hiện phép biên duyên hóa: cấp chuỗi và cấp token.

Ví dụ minh họa phép biên duyên hóa

Để làm rõ sự khác biệt giữa hai biến thể, xét truy vấn x: "Thủ đô của Pháp là gì và thành phố đó nằm bên sông nào?". Giả sử bộ truy xuất DPR trả về hai đoạn văn bản: z_1 chứa thông tin "Paris là thủ đô của Pháp", trong khi z_2 chứa thông tin "Paris nằm bên sông Seine". Trong ví dụ này, trọng số truy xuất được giả định đã chuẩn hóa trên tập hai tài liệu lần lượt là $p_\eta(z_1 \mid x) = 0.7$ và $p_\eta(z_2 \mid x) = 0.3$. Các giá trị xác suất trong ví dụ chỉ mang tính minh họa, không phải kết quả thực nghiệm từ mô hình BART hoặc DPR.

Để đơn giản hóa phép tính, xét chuỗi đầu ra gồm hai token đại diện cho hai thông tin cần trả lời, $y = (y_1, y_2) = (\text{Paris}, \text{Seine})$. Đây là cách rút gọn để minh họa công thức, không phản ánh quá trình tách token của một câu trả lời tự nhiên trong BART. Giả sử Generator tính được:

$$\begin{aligned} p_\theta(y_1 \mid x, z_1) &= 0.9, & p_\theta(y_1 \mid x, z_2) &= 0.4, \\ p_\theta(y_2 \mid x, z_1, y_1) &= 0.2, & p_\theta(y_2 \mid x, z_2, y_1) &= 0.8. \end{aligned} \tag{3}$$

Trường hợp RAG-Sequence

Với RAG-Sequence, xác suất sinh toàn bộ chuỗi được tính riêng trên từng tài liệu trước khi kết hợp với trọng số truy xuất. Trên z_1, xác suất chuỗi mục tiêu bằng $0.9 \times 0.2 = 0.18$; tương tự, trên z_2 giá trị này là $0.4 \times 0.8 = 0.32$. Áp dụng phép biên duyên hóa trong (1), ta có:

$$\begin{aligned} p_{\text{Seq}}(y \mid x) &\approx 0.7(0.9 \times 0.2) \\ &\quad + 0.3(0.4 \times 0.8) \\ &= 0.126 + 0.096 = 0.222. \end{aligned} \tag{4}$$

Giá trị 0.222 biểu thị xác suất mô hình gán cho chuỗi mục tiêu sau khi tổng hợp đóng góp từ hai tài liệu. Cần lưu ý rằng RAG-Sequence không đơn thuần lựa chọn tài liệu có trọng số truy xuất lớn nhất. Mỗi tài liệu đều tham gia vào phép tổng, nhưng toàn bộ chuỗi được điều kiện hóa trên một tài liệu cố định trong từng thành phần của phép tính [1].

Trường hợp RAG-Token

Đối với RAG-Token, phép biên duyên hóa được thực hiện tại từng vị trí token. Trước hết, xác suất tổng hợp để sinh token $y_1 = \text{Paris}$ được tính như sau:

$$\begin{aligned} p_{\text{Tok}}(y_1 \mid x) &\approx 0.7 \times 0.9 + 0.3 \times 0.4 \\ &= 0.75. \end{aligned} \tag{5}$$

Sau khi xét tiền tố $y_1 = \text{Paris}$, xác suất sinh token tiếp theo $y_2 = \text{Seine}$ là:

$$\begin{aligned} p_{\text{Tok}}(y_2 \mid x, y_1) &\approx 0.7 \times 0.2 + 0.3 \times 0.8 \\ &= 0.38. \end{aligned} \tag{6}$$

Xác suất của toàn bộ chuỗi vì vậy được tính bằng tích của hai xác suất token đã tổng hợp:

$$\begin{aligned} p_{\mathrm{Tok}}(y \mid x) &\approx 0.75 \times 0.38 \\ &= 0.285. \end{aligned} \tag{7}$$

Trong phép tính này, tài liệu z_1 đóng góp 0.63 vào xác suất sinh token đầu tiên, trong khi z_2 đóng góp 0.24 vào xác suất sinh token thứ hai, lớn hơn mức 0.14 của z_1. Điều đó cho thấy mức đóng góp của mỗi tài liệu có thể thay đổi theo vị trí token. Tuy nhiên, đây vẫn là sự kết hợp xác suất từ các tài liệu, không phải thao tác chọn cứng một tài liệu cho từng token.

Với cùng các xác suất giả định, RAG-Sequence cho kết quả 0.222 còn RAG-Token cho kết quả 0.285. Sự khác biệt xuất phát từ thứ tự của phép tổng và phép nhân: RAG-Sequence tính xác suất của cả chuỗi trên từng tài liệu rồi mới lấy tổng có trọng số, trong khi RAG-Token lấy tổng có trọng số ở mỗi vị trí trước khi nhân các xác suất token. Giá trị cao hơn trong ví dụ không đồng nghĩa với việc một biến thể luôn tạo ra câu trả lời tốt hơn biến thể còn lại [1].

B. Quy trình huấn luyện của mô hình RAG (Training)

Trong thiết kế nguyên bản, Lewis *et al.* huấn luyện đồng thời Retriever và Generator mà không cần nhãn giám sát trực tiếp chỉ rõ tài liệu nào phải được truy xuất. Với tập dữ liệu gồm các cặp đầu vào và đầu ra mục tiêu (x_j, y_j), mô hình cực tiểu hóa hàm mất mát Negative Marginal Log-Likelihood, được biểu diễn như sau [1]:

$$\mathcal{L}(\eta, \theta) = -\sum_{j=1}^{M} \log p_{\eta,\theta}(y_j \mid x_j), \tag{8}$$

trong đó M là số mẫu huấn luyện và $p_{\eta,\theta}(y_j \mid x_j)$ được tính theo công thức của RAG-Sequence hoặc RAG-Token. Dấu âm trước hàm logarit khiến Loss giảm khi xác suất gán cho câu trả lời chuẩn tăng. Do đó, việc cực tiểu hóa (8) tương ứng với việc tăng xác suất của các đầu ra mục tiêu. Nếu minh họa bằng một bước gradient descent cơ bản với tốc độ học α, các tham số có thể được cập nhật theo biểu thức:

$$\eta \leftarrow \eta - \alpha \nabla_{\eta} \mathcal{L}$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}$$

Áp dụng cho ví dụ trên với một mẫu huấn luyện, nếu chuỗi $(\mathrm{Paris}, \mathrm{Seine})$ là đáp án chuẩn, giá trị Loss lần lượt là:

$$\begin{aligned} \mathcal{L}_{\mathrm{Seq}} &= -\ln(0.222) \approx 1.505, \\ \mathcal{L}_{\mathrm{Tok}} &= -\ln(0.285) \approx 1.255. \end{aligned} \tag{9}$$

Đây là các giá trị minh họa trên cùng một mẫu, không phải phép so sánh thực nghiệm giữa hai biến thể. Công thức cập nhật gradient ở trên chỉ minh họa hướng tối ưu; Adam thực tế sử dụng thêm các ước lượng mô-men bậc nhất và bậc hai.

Trong mỗi bước huấn luyện, DPR truy xuất tập top-K tài liệu theo truy vấn đầu vào. BART nhận truy vấn kết hợp với từng tài liệu để tính xác suất sinh các token thuộc câu trả lời chuẩn. Sau khi biên duyên hóa, mô hình tính Loss và sử dụng lan truyền ngược (*backpropagation*) để tính gradient, sau đó cập nhật các tham số bằng bộ tối ưu Adam [4]. Các token mục tiêu đã biết được sử dụng làm tiền tố khi tính xác suất trong quá trình huấn luyện; mô hình không cần tự lựa chọn từng token đầu ra để tính Loss.

Vì Loss phụ thuộc đồng thời vào trọng số truy xuất và xác suất sinh văn bản, tín hiệu học từ câu trả lời chuẩn có thể điều chỉnh cả bộ truy xuất lẫn bộ sinh, dù tập huấn luyện không cung cấp nhãn tài liệu liên quan. Trong thực nghiệm của Lewis *et al.*, bộ mã hóa truy vấn BERT_q của DPR và Generator BART được tinh chỉnh; bộ mã hóa tài liệu BERT_d cùng chỉ mục vector được giữ cố định nhằm tránh chi phí tính toán và cập nhật lại chỉ mục trong quá trình huấn luyện [1].

C. Quy trình giải mã của mô hình RAG (Decoding)

Khác với huấn luyện, ở giai đoạn suy luận mô hình không có sẵn câu trả lời chuẩn. Bài toán đặt ra là tìm chuỗi đầu ra có xác suất cao theo mô hình, hay xấp xỉ nghiệm của $y^* = \arg\max_y p(y \mid x)$. Do cấu trúc biên duyên hóa khác nhau, RAG-Token và RAG-Sequence sử dụng hai phương pháp giải mã riêng [1].

a) Giải mã RAG-Token: Tại mỗi vị trí, mô hình tổng hợp phân bố xác suất token từ các tài liệu truy xuất:

$$\begin{aligned} p_{\mathrm{Tok}}(y_i \mid x, y_{<i}) = \sum_{z \in \mathcal{Z}_K(x)} & p_{\eta}(z \mid x) \\ & \cdot p_{\theta}(y_i \mid x, z, y_{<i}). \end{aligned} \tag{10}$$

Phân bố này được đưa vào thuật toán Beam Search để mở rộng các chuỗi ứng viên. Quá trình tiếp tục theo cơ chế tự hồi quy, trong đó các token đã sinh trở thành tiền tố cho bước dự đoán tiếp theo. Chẳng hạn, trong ví dụ minh họa, xác suất tổng hợp của token "Paris" là 0.75; nếu chuỗi ứng viên được mở rộng bằng token "Seine" với xác suất có điều kiện 0.38, điểm xác suất của chuỗi hai token là 0.285. Trên thực tế, Beam Search còn xét những token và chuỗi ứng viên khác, thay vì chỉ dựa trên một đường đi duy nhất [1].

b) Giải mã RAG-Sequence: Đối với RAG-Sequence, xác suất của toàn bộ chuỗi không phân rã thành phân bố chuyển tiếp token theo cách thông thường. Vì vậy, không thể áp dụng trực tiếp một lần Beam Search chung như RAG-Token. Thay vào đó, Beam Search được thực hiện riêng trên từng tài liệu để tạo tập chuỗi ứng viên. Mỗi ứng viên sau đó được đánh giá bằng cách kết hợp xác suất sinh trọn chuỗi trên các tài liệu với trọng số truy xuất tương ứng [1].

Xét thêm ứng viên $y' = (\mathrm{Paris}, \mathrm{Rhône})$, với xác suất sinh token thứ hai trên z_1 và z_2 lần lượt là 0.1 và 0.05. Giữ nguyên xác suất token đầu, ta có:

$$\begin{aligned} p_{\mathrm{Seq}}(y' \mid x) &\approx 0.7(0.9 \times 0.1) \\ &\quad + 0.3(0.4 \times 0.05) = 0.069, \\ p_{\mathrm{Tok}}(y' \mid x) &\approx 0.75(0.7 \times 0.1 \\ &\quad + 0.3 \times 0.05) = 0.06375. \end{aligned} \tag{11}$$

Như vậy, trong hai ứng viên được xét, cả hai biến thể đều ưu tiên $(\mathrm{Paris}, \mathrm{Seine})$. Điều này không bảo đảm đó là chuỗi tối ưu trên toàn bộ không gian đầu ra.

Tóm lại, $p(y \mid x)$ được dùng để tính Loss và cập nhật tham số khi huấn luyện, đồng thời làm cơ sở đánh giá các chuỗi ứng viên khi giải mã. Do thực hiện biên duyên hóa ở hai cấp độ khác nhau, RAG-Sequence và RAG-Token cũng đòi hỏi cách tổ chức giải mã khác nhau [1].

D. Phương pháp Thorough Decoding và Fast Decoding

Đối với RAG-Sequence, xác suất của một chuỗi đầu ra được tính bằng cách lấy tổng có trọng số trên các tài liệu truy xuất, như đã trình bày trong (1). Do phép biên duyên hóa diễn ra ở cấp chuỗi, mô hình không thể trực tiếp áp dụng một lần Beam Search thông thường trên phân bố xác suất token đã tổng hợp như RAG-Token. Thay vào đó, Lewis *et al.* đề xuất thực hiện Beam Search riêng trên từng tài liệu, sau đó đánh giá lại các câu trả lời ứng viên theo xác suất của toàn bộ chuỗi [1].

Gọi $\mathcal{Z}_K(x) = \{z_1, \ldots, z_K\}$ là tập các tài liệu được truy xuất từ truy vấn x, và $\mathcal{B}(z)$ là tập các chuỗi ứng viên thu được khi thực hiện Beam Search với ngữ cảnh z. Tập ứng viên chung được xây dựng bằng phép hợp:

$$\mathcal{Y} = \bigcup_{z \in \mathcal{Z}_K(x)} \mathcal{B}(z). \tag{12}$$

Mỗi chuỗi $y \in \mathcal{Y}$ có thể xuất hiện trong danh sách ứng viên của một số tài liệu nhưng không xuất hiện ở các tài liệu còn lại. Hai phương pháp dưới đây khác nhau ở cách xử lý những trường hợp đó.

a) Thorough Decoding (Giải mã đánh giá đầy đủ): Phương pháp Thorough Decoding tính điểm cho mỗi chuỗi ứng viên bằng cách xét đóng góp của toàn bộ K tài liệu truy xuất, kể cả những tài liệu không tạo ra chuỗi đó trong quá trình Beam Search. Xác suất sinh chuỗi y khi điều kiện hóa trên tài liệu z được xác định theo quy tắc nhân xác suất có điều kiện:

$$p_\theta(y \mid x, z) = \prod_{i=1}^{|y|} p_\theta(y_i \mid x, z, y_{1:i-1}). \tag{13}$$

Với những cặp (y, z) đã xuất hiện trong Beam Search, mô hình có thể sử dụng xác suất sinh chuỗi đã tính. Ngược lại, nếu $y \notin \mathcal{B}(z)$, Generator cần thực hiện thêm một lượt truyền xuôi (*forward pass*) để tính $p_\theta(y \mid x, z)$ cho chuỗi ứng viên đó. Điểm của y sau khi biên duyên hóa trên top-K được viết:

$$s_{\mathrm{Th}}(y \mid x) = \sum_{z \in \mathcal{Z}_K(x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z). \tag{14}$$

Sau khi tính điểm cho các ứng viên thuộc $\mathcal{Y}$, mô hình chọn câu trả lời có điểm cao nhất:

$$\hat{y}_{\mathrm{Th}} = \arg\max_{y \in \mathcal{Y}} s_{\mathrm{Th}}(y \mid x). \tag{15}$$

Cách đánh giá này không bỏ qua đóng góp của tài liệu chỉ vì một chuỗi vắng mặt trong beam tương ứng. Tuy nhiên, khi câu trả lời dài hoặc số tài liệu truy xuất lớn, tập $\mathcal{Y}$ có thể chứa nhiều ứng viên, kéo theo nhiều lượt truyền xuôi bổ sung. Vì vậy, Thorough Decoding có chi phí tính toán cao hơn, dù vẫn chỉ tìm kiếm trong tập ứng viên $\mathcal{Y}$ chứ không xét mọi chuỗi có thể có [1].

b) Fast Decoding (Giải mã nhanh): Để giảm số lượt tính toán, Fast Decoding sử dụng cùng cách tạo tập ứng viên $\mathcal{Y}$ nhưng không thực hiện thêm lượt truyền xuôi cho những cặp (y, z) chưa xuất hiện trong Beam Search. Thay vào đó, xác suất sinh chuỗi được xấp xỉ bằng 0 nếu chuỗi y không nằm trong danh sách ứng viên của tài liệu z [1]:

$$\widetilde{p}_\theta(y \mid x, z) = \begin{cases} p_\theta(y \mid x, z), & y \in \mathcal{B}(z), \\ 0, & y \notin \mathcal{B}(z). \end{cases} \tag{16}$$

Từ đó, điểm tổng hợp được tính bởi:

$$s_{\mathrm{Fast}}(y \mid x) = \sum_{z \in \mathcal{Z}_K(x)} p_\eta(z \mid x)\, \widetilde{p}_\theta(y \mid x, z), \tag{17}$$

và chuỗi có điểm cao nhất trong tập ứng viên được lựa chọn làm câu trả lời:

$$\hat{y}_{\mathrm{Fast}} = \arg\max_{y \in \mathcal{Y}} s_{\mathrm{Fast}}(y \mid x). \tag{18}$$

Cần lưu ý rằng $\widetilde{p}_\theta(y \mid x, z) = 0$ là một quy ước xấp xỉ để giảm chi phí giải mã, không có nghĩa Generator thực sự gán xác suất bằng 0 cho chuỗi đó. Do loại bỏ các đóng góp chưa được tính, Fast Decoding có thể làm thay đổi điểm số và thứ tự của các ứng viên so với Thorough Decoding.

c) Ví dụ minh họa hai phương pháp: Xét hai tài liệu z_1, z_2 có trọng số truy xuất lần lượt là $p_\eta(z_1 \mid x) = 0.7$ và $p_\eta(z_2 \mid x) = 0.3$. Giả sử tập ứng viên có hai chuỗi cần so sánh: $y_A =$ “Paris, Seine” và $y_B =$ “Paris, Rhône”. Chuỗi y_A xuất hiện trong beam của cả hai tài liệu, trong khi y_B chỉ xuất hiện trong beam của z_1. Các xác suất sinh chuỗi giả định lần lượt là:

$$\begin{aligned} &p_\theta(y_A \mid x, z_1) = 0.18, \quad p_\theta(y_A \mid x, z_2) = 0.32, \\ &p_\theta(y_B \mid x, z_1) = 0.09, \quad p_\theta(y_B \mid x, z_2) = 0.02. \end{aligned} \tag{19}$$

Với Thorough Decoding, mô hình tính cả xác suất $p_\theta(y_B \mid x, z_2) = 0.02$ thông qua lượt truyền xuôi bổ sung dù y_B không xuất hiện trong beam của z_2:

$$\begin{aligned} s_{\mathrm{Th}}(y_A \mid x) &= 0.7(0.18) + 0.3(0.32) = 0.222, \\ s_{\mathrm{Th}}(y_B \mid x) &= 0.7(0.09) + 0.3(0.02) = 0.069. \end{aligned} \tag{20}$$

Đối với Fast Decoding, xác suất của y_B trên z_2 được xấp xỉ bằng 0, còn điểm của y_A không thay đổi vì chuỗi này xuất hiện trong cả hai beam:

$$\begin{aligned} s_{\mathrm{Fast}}(y_A \mid x) &= 0.7(0.18) + 0.3(0.32) = 0.222, \\ s_{\mathrm{Fast}}(y_B \mid x) &= 0.7(0.09) + 0.3(0) = 0.063. \end{aligned} \tag{21}$$

Như vậy, cả hai phương pháp đều đánh giá y_A cao hơn y_B trong ví dụ này, nhưng điểm của y_B khác nhau do Fast Decoding bỏ qua đóng góp từ z_2. Các giá trị trên chỉ được sử dụng để minh họa phép tính, không phải kết quả thực nghiệm. Về bản chất, Thorough Decoding đánh giá đầy đủ hơn xác suất của từng ứng viên trên top-K, trong khi Fast Decoding giảm chi phí bằng một phép xấp xỉ bổ sung; cả hai vẫn phụ thuộc vào những ứng viên được tìm thấy bởi Beam Search [1].

III. Sự tiến hóa của các mô hình RAG

Từ kiến trúc truy xuất rồi sinh văn bản ban đầu, RAG đã được phát triển theo hướng cải thiện chất lượng ngữ cảnh và tổ chức linh hoạt hơn quá trình truy xuất. Gao *et al.* phân chia sự phát triển này thành ba mô hình chính: Naive RAG, Advanced RAG và Modular RAG. Đây là cách phân loại theo cấu trúc và khả năng xử lý của hệ thống, không phải ba thế hệ hoàn toàn tách biệt về thời gian. Trong đó, Naive RAG cung cấp quy trình nền tảng, Advanced RAG tập trung xử lý các hạn chế của quy trình truy xuất tuyến tính, còn Modular RAG cho phép phối hợp nhiều thành phần và luồng xử lý tùy theo yêu cầu của bài toán [2].

A. Naive RAG (RAG truyền thống)

Naive RAG là dạng triển khai cơ bản của kiến trúc tạo sinh tăng cường truy xuất, thường được mô tả bằng quy trình *Retrieve–Read*. Hệ thống lần lượt xây dựng chỉ mục tài liệu, truy xuất thông tin liên quan đến câu hỏi và cung cấp những thông tin đó cho mô hình ngôn ngữ để tạo câu trả lời. Ba giai đoạn chính là Indexing, Retrieval và Generation [2].

a) Indexing (Xây dựng chỉ mục): Ở bước đầu tiên, nguồn dữ liệu bên ngoài như văn bản, trang web hoặc tài liệu PDF được thu thập và chuyển về định dạng phù hợp để xử lý. Hệ thống thường làm sạch dữ liệu, loại bỏ những nội dung không cần thiết rồi chia tài liệu thành các đoạn nhỏ (*chunks*). Việc phân đoạn giúp hạn chế độ dài của ngữ cảnh được đưa vào mô hình ngôn ngữ và tạo ra các đơn vị thông tin có thể truy xuất độc lập. Mỗi đoạn sau đó được mã hóa thành vector đặc trưng (*embedding*) và lưu trong chỉ mục tìm kiếm vector. Chất lượng phân đoạn có ảnh hưởng trực tiếp đến khả năng giữ lại đầy đủ nội dung của từng đơn vị thông tin [2].

b) Retrieval (Truy xuất): Khi nhận câu hỏi x, bộ truy xuất mã hóa câu hỏi thành một vector và so sánh với các vector tài liệu đã được lập chỉ mục. Hệ thống chọn ra K đoạn có điểm tương đồng cao, tạo thành tập ngữ cảnh $\mathcal{Z}_K(x)$. Có thể biểu diễn bước lựa chọn này dưới dạng:

$$\mathcal{Z}_K(x) = \underset{z \in \mathcal{D}}{\mathrm{TopK}} \left[\mathrm{sim}(\mathbf{q}(x), \mathbf{d}(z)) \right], \tag{22}$$

trong đó $\mathcal{D}$ là kho tài liệu, $\mathbf{q}(x)$ và $\mathbf{d}(z)$ lần lượt là biểu diễn vector của truy vấn và đoạn văn bản, còn sim là hàm đánh giá độ tương đồng. Tùy bộ truy xuất, hàm này có thể sử dụng tích vô hướng hoặc độ tương đồng cosine. Các đoạn thuộc top-K được chuyển sang giai đoạn tạo sinh [2].

c) Generation (Sinh câu trả lời): Mô hình ngôn ngữ nhận câu hỏi gốc kết hợp với các đoạn văn bản truy xuất để xây dựng câu trả lời. Trong cách triển khai *Retrieve–Read* thông thường, ngữ cảnh được ghép vào lời nhắc (*prompt*) và đưa trực tiếp cho mô hình. Bằng việc tham khảo tri thức bên ngoài, mô hình có thêm căn cứ để xử lý những câu hỏi mà bộ nhớ tham số chưa chứa đủ thông tin. Tuy nhiên, kết quả vẫn phụ thuộc vào mức độ liên quan và chất lượng của những đoạn đã truy xuất, cũng như khả năng sử dụng ngữ cảnh của mô hình sinh [2].

Những hạn chế của Naive RAG Mặc dù quy trình ba bước tương đối dễ xây dựng, Naive RAG còn tồn tại một số hạn chế. Trước hết, ở giai đoạn truy xuất, việc lựa chọn chỉ dựa trên điểm tương đồng có thể trả về các đoạn văn bản liên quan về từ ngữ nhưng chưa đáp ứng đúng ý định của câu hỏi. Ngược lại, những đoạn chứa thông tin cần thiết có thể không xuất hiện trong top-K, làm giảm độ bao phủ thông tin (*recall*). Nếu dữ liệu bị chia tại những vị trí không phù hợp, một sự kiện hoặc lập luận còn có thể bị tách sang nhiều đoạn, khiến mỗi đoạn riêng lẻ thiếu ngữ cảnh cần thiết [2].

Bên cạnh đó, việc chuyển toàn bộ những đoạn truy xuất vào Generator có thể đưa vào nhiều thông tin trùng lặp, nhiễu hoặc mâu thuẫn. Ngữ cảnh quá dài không đồng nghĩa với câu trả lời chính xác hơn; mô hình vẫn có thể bỏ qua chứng cứ quan trọng, tạo ra thông tin không được tài liệu hỗ trợ hoặc lặp lại nội dung được truy xuất. Đối với câu hỏi phức tạp cần nhiều bước tìm kiếm và suy luận, một lượt truy xuất dựa trên câu hỏi ban đầu cũng có thể không cung cấp đủ dữ kiện. Những hạn chế này là cơ sở để các phương pháp Advanced RAG và Modular RAG được phát triển [2].

B. Advanced RAG (RAG nâng cao)

Advanced RAG tiếp tục sử dụng các thành phần truy xuất và tạo sinh, nhưng bổ sung những kỹ thuật tối ưu hóa trước và sau bước truy xuất. Mục tiêu không chỉ là tìm được tài liệu có độ tương đồng cao mà còn bảo đảm ngữ cảnh chuyển cho Generator có đủ thông tin, ít nhiễu và phù hợp với yêu cầu của truy vấn. Hai nhóm kỹ thuật được sử dụng phổ biến là *Pre-retrieval Optimization* và *Post-retrieval Processing* [2].

1) Tối ưu hóa trước truy xuất (Pre-retrieval Optimization): Ở giai đoạn trước truy xuất, Advanced RAG cải thiện cả cách tổ chức kho tài liệu lẫn cách biểu diễn câu hỏi. Đối với kho tri thức, chiến lược phân đoạn (*chunking*) được điều chỉnh theo độ dài và cấu trúc nội dung thay vì chỉ chia văn bản theo một kích thước cố định. Các phương pháp như phân đoạn có phần chồng lấn (*overlap*), cửa sổ trượt (*sliding window*) hoặc kết hợp đoạn nhỏ với ngữ cảnh lớn hơn (*Small-to-Big*) giúp hạn chế mất mát thông tin tại ranh giới giữa các đoạn. Tuy vậy, đoạn quá nhỏ có thể thiếu ý nghĩa đầy đủ, còn đoạn quá lớn lại dễ mang theo thông tin không liên quan. Vì thế, kích thước và mức độ chi tiết của đoạn cần được lựa chọn theo đặc điểm dữ liệu và tác vụ [2].

Ngoài nội dung văn bản, các đoạn có thể được gắn thêm siêu dữ liệu (*metadata*) như nguồn tài liệu, tiêu đề, thời gian tạo, số trang hoặc lĩnh vực. Metadata hỗ trợ lọc và giới hạn phạm vi truy xuất trước khi tính điểm tương đồng, đặc biệt khi kho tri thức có nhiều chủ đề hoặc dữ liệu được cập nhật thường xuyên. Những cách lập chỉ mục theo cấu trúc phân cấp cũng cho phép liên kết giữa đoạn văn bản và tài liệu gốc, nhờ đó có thể truy xuất ở độ chi tiết phù hợp mà vẫn giữ được ngữ cảnh rộng hơn [2].

Đối với truy vấn, Advanced RAG áp dụng các kỹ thuật *Query Rewriting* và *Query Expansion*. Query Rewriting viết lại câu hỏi thành biểu thức rõ ràng hơn, chẳng hạn xử lý từ viết tắt hoặc câu hỏi thiếu ngữ cảnh mà không thay đổi mục tiêu thông tin. Trong khi đó, Query Expansion mở rộng truy vấn thành nhiều cách diễn đạt hoặc câu hỏi liên quan để tăng khả năng tìm được các tài liệu mang thông tin cần thiết. Các

phương pháp như phân tách câu hỏi phức tạp thành các truy vấn con (*sub-queries*) hoặc tạo văn bản giả định làm cơ sở truy xuất (*HyDE*) cũng được khảo sát trong nhóm kỹ thuật này. Việc biến đổi truy vấn cần duy trì ý định gốc, bởi nếu mở rộng không phù hợp, hệ thống có thể truy xuất thêm những đoạn không liên quan [2].

2) Xử lý sau truy xuất (Post-retrieval Processing): Sau khi thu được tập tài liệu ban đầu, Advanced RAG thực hiện thêm bước lựa chọn và tổ chức lại ngữ cảnh trước khi đưa vào Generator. Một kỹ thuật thường dùng là xếp hạng lại (*Reranking*), trong đó các đoạn được đánh giá bằng mô hình hoặc tiêu chí xếp hạng phù hợp hơn với câu hỏi. Reranking có thể thay đổi thứ tự ưu tiên, loại bỏ những đoạn có giá trị thấp và giữ lại các đoạn có khả năng hỗ trợ câu trả lời tốt hơn. Khác với tìm kiếm vector ban đầu vốn chú trọng tốc độ trên kho dữ liệu lớn, bước này thường tập trung vào chất lượng đánh giá trên một tập ứng viên nhỏ hơn [2].

Kỹ thuật còn lại là lựa chọn và nén ngữ cảnh (*Context Selection/Compression*). Thay vì đưa nguyên văn toàn bộ các đoạn truy xuất vào mô hình ngôn ngữ, hệ thống tìm những phần thông tin quan trọng, loại bỏ nội dung trùng lặp hoặc rút gọn văn bản sao cho vẫn giữ lại chứng cứ cần thiết. Việc này vừa giảm độ dài đầu vào và chi phí xử lý, vừa hạn chế ảnh hưởng của những đoạn nhiễu. Tuy nhiên, nếu nén quá mức, các điều kiện, mốc thời gian hoặc quan hệ giữa các sự kiện có thể bị mất, dẫn đến suy giảm độ chính xác của câu trả lời [2].

Một vấn đề liên quan là hiện tượng *Lost in the Middle*: mô hình ngôn ngữ có thể khai thác kém hơn thông tin nằm ở phần giữa của ngữ cảnh dài so với thông tin ở đầu hoặc cuối. Vì vậy, xếp hạng lại và nén ngữ cảnh có thể giúp tổ chức chứng cứ quan trọng ở vị trí phù hợp và giảm lượng văn bản không cần thiết. Dù vậy, các phương pháp này chỉ góp phần giảm tác động của ngữ cảnh dài, không bảo đảm loại bỏ hoàn toàn hiện tượng trên [2].

Nhìn chung, Advanced RAG cải thiện chất lượng của từng giai đoạn trong cùng một quy trình truy xuất–tạo sinh tương đối tuyến tính. Khi bài toán cần thay đổi chiến lược tìm kiếm trong quá trình trả lời hoặc phối hợp nhiều nguồn tri thức, một kiến trúc mô-đun hóa sẽ phù hợp hơn [2].

C. Modular RAG (RAG mô-đun hóa)

Modular RAG kế thừa các kỹ thuật của hai mô hình trước nhưng thay đổi cách tổ chức hệ thống. Thay vì phụ thuộc vào một chuỗi xử lý cố định, kiến trúc này chia quy trình RAG thành những mô-đun có nhiệm vụ riêng và cho phép kết hợp, thay thế hoặc gọi lại các mô-đun theo đặc điểm của truy vấn. Nhờ đó, hệ thống có thể lựa chọn nguồn dữ liệu, điều chỉnh cách truy xuất và thực hiện nhiều lượt xử lý trước khi đưa ra câu trả lời cuối cùng [2].

1) Các mô-đun mở rộng: Bên cạnh Retriever và Generator, Gao *et al.* mô tả một số mô-đun có thể bổ sung trong Modular RAG [2]:

- **Search:** Thực hiện tìm kiếm trên những nguồn thông tin phù hợp với truy vấn, có thể bao gồm công cụ tìm kiếm, cơ sở dữ liệu hoặc đồ thị tri thức. Mô-đun này mở rộng phương thức tiếp cận dữ liệu thay vì chỉ phụ thuộc vào một chỉ mục vector cố định.
- **Memory:** Lưu giữ và tái sử dụng thông tin từ những lần truy xuất hoặc sinh văn bản trước đó. Thông tin đã thu thập có thể hỗ trợ các bước xử lý tiếp theo, đặc biệt trong những quy trình cần nhiều lượt truy xuất.
- **Routing:** Phân tích đặc điểm câu hỏi để lựa chọn nguồn dữ liệu, công cụ hoặc luồng xử lý thích hợp. Chẳng hạn, các yêu cầu tìm kiếm dữ kiện, tổng hợp nội dung và truy vấn cơ sở dữ liệu có thể được chuyển qua các nhánh xử lý khác nhau.
- **Predict:** Sử dụng khả năng sinh của mô hình ngôn ngữ để tạo thông tin hoặc ngữ cảnh trung gian hỗ trợ các bước tiếp theo, qua đó giảm sự phụ thuộc vào một quy trình truy xuất trực tiếp duy nhất.
- **Task Adapter:** Điều chỉnh cách chuẩn bị đầu vào, chọn thông tin hỗ trợ hoặc cấu hình bước xử lý theo tác vụ cụ thể, giúp cùng một hệ thống thích nghi với nhiều yêu cầu khác nhau.

Các mô-đun trên không bắt buộc phải xuất hiện đồng thời trong mọi hệ thống. Việc lựa chọn và sắp xếp phụ thuộc vào mục tiêu ứng dụng, nguồn dữ liệu sẵn có và chi phí tính toán chấp nhận được [2].

2) Các luồng xử lý mới: Iterative, Recursive và Adaptive: Không chỉ bổ sung thành phần, Modular RAG còn cho phép thay đổi trình tự phối hợp giữa truy xuất và sinh văn bản. Ba luồng xử lý tiêu biểu là truy xuất lặp, truy xuất đệ quy và truy xuất thích ứng [2].

a) Iterative Retrieval (Truy xuất lặp): Ở luồng truy xuất lặp, quá trình truy xuất và tạo sinh được thực hiện xen kẽ qua nhiều vòng. Sau lượt truy xuất đầu tiên, kết quả hoặc nội dung mà mô hình đã tạo ra được sử dụng để hình thành truy vấn tiếp theo. Cách làm này cho phép bổ sung dần những thông tin còn thiếu và điều chỉnh hướng tìm kiếm theo tiến trình giải quyết câu hỏi. Chẳng hạn, hệ thống ITER-RETGEN kết hợp kết quả sinh văn bản với quá trình truy xuất kế tiếp để cải thiện ngữ cảnh cho những vòng tạo sinh sau [2].

b) Recursive Retrieval (Truy xuất đệ quy): Truy xuất đệ quy phù hợp với câu hỏi phức tạp cần được chia thành những vấn đề nhỏ hơn. Hệ thống có thể phân rã câu hỏi ban đầu thành các truy vấn con, giải quyết từng phần rồi kết hợp kết quả. Ngoài ra, nếu kho tài liệu được tổ chức phân cấp, quá trình truy xuất có thể bắt đầu từ bản tóm tắt hoặc cấu trúc tổng quát, sau đó đi sâu vào những đoạn cụ thể. Điểm nổi bật của phương pháp này là khả năng tổ chức quá trình tìm kiếm theo quan hệ giữa bài toán chính và các bài toán thành phần [2].

c) Adaptive Retrieval (Truy xuất thích ứng): Khác với những hệ thống truy xuất theo số vòng cố định, luồng thích ứng cho phép mô hình quyết định có cần truy xuất thông tin bên ngoài hay không, khi nào cần truy xuất và lúc nào có thể tiếp tục tạo sinh. Những phương pháp như FLARE và Self-RAG minh họa hướng tiếp cận này bằng cách đánh giá nhu cầu thông tin trong quá trình sinh câu trả lời. Cơ chế truy xuất có điều kiện giúp tránh những lượt tìm kiếm không cần thiết,

đồng thời vẫn cho phép bổ sung chứng cứ ở những thời điểm mô hình chưa có đủ cơ sở để trả lời [2].

Việc mô-đun hóa làm tăng khả năng mở rộng và tùy biến của RAG, nhưng cũng đặt ra yêu cầu kiểm soát độ trễ, chi phí của nhiều lượt gọi mô hình và sai lệch có thể tích lũy qua các bước xử lý. Vì vậy, Modular RAG không nhất thiết phù hợp hơn trong mọi bài toán; một quy trình đơn giản vẫn có thể là lựa chọn hợp lý khi yêu cầu truy xuất và tạo sinh đã được xác định rõ [2].

D. So sánh RAG với Fine-tuning và Prompt Engineering

RAG, Fine-tuning và Prompt Engineering đều được sử dụng để cải thiện kết quả của mô hình ngôn ngữ, nhưng tác động lên hệ thống ở những cấp độ khác nhau. Theo Gao *et al.*, có thể xem xét ba hướng tiếp cận này theo hai khía cạnh: mức độ cần đến tri thức bên ngoài và mức độ cần điều chỉnh tham số mô hình [2].

a) Prompt Engineering (Thiết kế lời nhắc): Prompt Engineering tập trung vào cách diễn đạt yêu cầu, bổ sung chỉ dẫn hoặc đưa ví dụ trong lời nhắc để khai thác khả năng sẵn có của mô hình. Phương pháp này thường không thay đổi trọng số và không bắt buộc xây dựng một hệ thống truy xuất riêng. Ưu điểm là triển khai nhanh và dễ điều chỉnh cho những tác vụ có yêu cầu tương đối rõ ràng. Tuy nhiên, nếu chỉ sử dụng tri thức đã lưu trong tham số mà không cung cấp thêm dữ liệu, mô hình vẫn gặp khó khăn trước những câu hỏi đòi hỏi thông tin mới hoặc chuyên ngành hẹp [2].

b) Fine-tuning (Tinh chỉnh mô hình): Fine-tuning cập nhật một phần hoặc toàn bộ tham số của mô hình bằng dữ liệu phù hợp với nhiệm vụ đích. Nhờ đó, mô hình có thể thích nghi tốt hơn với cách diễn đạt, định dạng đầu ra hoặc đặc điểm của một lĩnh vực ứng dụng. Đổi lại, phương pháp này cần dữ liệu và tài nguyên huấn luyện; việc đưa những sự kiện mới vào mô hình cũng không thuận tiện bằng cập nhật một nguồn tri thức bên ngoài. Vì vậy, Fine-tuning thường phù hợp khi cần thay đổi hành vi hoặc năng lực xử lý của mô hình theo một nhiệm vụ ổn định [2].

c) Retrieval-Augmented Generation: Khác với hai phương pháp trên, RAG cung cấp dữ liệu tham chiếu tại thời điểm thực thi thông qua bộ truy xuất. Khi nội dung của kho tri thức thay đổi, hệ thống có thể cập nhật tài liệu hoặc chỉ mục mà không nhất thiết phải huấn luyện lại Generator. Điều này đặc biệt có ích đối với câu hỏi cần thông tin thường xuyên được bổ sung hoặc cần đối chiếu với nguồn dữ liệu cụ thể. Tuy nhiên, RAG làm tăng chi phí của bước tìm kiếm và xử lý ngữ cảnh; chất lượng câu trả lời còn phụ thuộc vào việc truy xuất đúng chứng cứ và cách mô hình sử dụng chứng cứ đó [2].

Ba hướng tiếp cận này không loại trừ lẫn nhau. Một hệ thống có thể sử dụng Prompt Engineering để quy định cách trả lời, Fine-tuning để cải thiện khả năng thực hiện tác vụ và RAG để bổ sung nguồn thông tin cần thiết. Việc lựa chọn hoặc kết hợp các phương pháp nên dựa trên yêu cầu về độ cập nhật tri thức, chất lượng đầu ra, khả năng kiểm chứng và ngân sách tính toán. Nhìn từ góc độ kiến trúc, sự chuyển dịch từ Naive RAG qua Advanced RAG đến Modular RAG cho thấy xu hướng không ngừng cải thiện cả chất lượng thông tin truy xuất lẫn cách tổ chức quá trình sử dụng thông tin đó [2].

IV. ĐỘT PHÁ VỚI RAG TỰ PHẢN BIỆN (SELF-RAG)

Trong mục này, d ký hiệu đoạn văn bản làm chứng cứ, y_t là phân đoạn đầu ra tại bước t và $y_{<t}$ là các phân đoạn đã sinh trước đó. Để phân biệt với tham số θ của BART trong RAG nguyên bản, tham số của Self-RAG Generator được ký hiệu ψ, còn tham số của Critic được ký hiệu ϕ.

Trong các kiến trúc RAG truyền thống, tài liệu thường được truy xuất theo một số lượng hoặc thời điểm định trước, sau đó cung cấp cho mô hình ngôn ngữ để tạo câu trả lời. Cách xử lý này có thể cải thiện khả năng tiếp cận tri thức bên ngoài, nhưng không bảo đảm rằng mỗi lần truy xuất đều cần thiết hoặc các thông tin được tìm thấy thực sự hỗ trợ nội dung đầu ra. Asai *et al.* đề xuất *Self-Reflective Retrieval-Augmented Generation* (Self-RAG), trong đó mô hình học cách tự quyết định nhu cầu truy xuất, sinh câu trả lời và đánh giá mức độ phù hợp của chứng cứ thông qua các token đặc biệt gọi là *Reflection Tokens* [3]. Điểm khác biệt của Self-RAG không chỉ nằm ở việc truy xuất linh hoạt mà còn ở khả năng đưa tín hiệu tự đánh giá vào quá trình giải mã, cho phép điều chỉnh cách lựa chọn câu trả lời theo yêu cầu của tác vụ.

A. Hạn chế của việc truy xuất thụ động

Trong nhiều hệ thống RAG, bộ truy xuất được gọi ngay khi nhận truy vấn và trả về một số đoạn văn bản cố định, bất kể nội dung câu hỏi có đòi hỏi kiến thức từ bên ngoài hay không. Asai *et al.* gọi đây là vấn đề truy xuất không phân biệt nhu cầu (*indiscriminate retrieval*) [3]. Chẳng hạn, câu hỏi về một dữ kiện ít phổ biến cần có nguồn kiểm chứng, trong khi yêu cầu viết một đoạn văn sáng tạo có thể được mô hình đáp ứng bằng tri thức và khả năng ngôn ngữ sẵn có. Nếu hai tác vụ đều bắt buộc truy xuất cùng số tài liệu, hệ thống phát sinh chi phí không cần thiết ở trường hợp thứ hai.

Bên cạnh chi phí và độ trễ, các đoạn truy xuất có thể chỉ trùng khớp về chủ đề nhưng không chứa chứng cứ cần thiết, hoặc thậm chí mang thông tin không liên quan. Khi Generator sử dụng trực tiếp những đoạn này, câu trả lời có nguy cơ bị ảnh hưởng bởi nhiễu ngữ cảnh. Ngay cả khi tài liệu được truy xuất đúng, mô hình vẫn có thể tạo ra nhận định không được chứng cứ hỗ trợ. Do đó, việc tăng số lượng tài liệu truy xuất chưa đủ để bảo đảm độ trung thực của đầu ra; hệ thống còn cần kiểm tra tính liên quan của tài liệu và mức độ hỗ trợ đối với từng phần nội dung được sinh [3].

Self-RAG giải quyết hai vấn đề trên theo hướng đưa quyết định truy xuất và đánh giá chứng cứ vào chính tiến trình tạo sinh. Thay vì xem Retriever là bước xử lý bắt buộc ở đầu quy trình, mô hình có thể gọi công cụ truy xuất theo nhu cầu, sau đó tự đánh giá các đoạn văn bản và câu trả lời ứng viên trước khi tiếp tục [3].

B. Cơ chế truy xuất thích ứng (Adaptive Retrieval)

Self-RAG tổ chức quá trình sinh văn bản thành nhiều phân đoạn (*segments*), chẳng hạn một câu hoặc một phần câu trả lời. Tại mỗi thời điểm, mô hình sử dụng truy vấn gốc x cùng

nội dung đã sinh $y_{<t}$ để dự đoán một token điều khiển truy xuất. Nhóm token *Retrieve* có ba lựa chọn về mặt chức năng: thực hiện truy xuất (*Yes*), không truy xuất (*No*) hoặc tiếp tục sử dụng chứng cứ đã có (*Continue*) [3].

Nếu mô hình đánh giá rằng cần có nguồn thông tin bổ sung, Retriever được gọi để tìm những đoạn liên quan, có thể dựa trên truy vấn và trạng thái tạo sinh hiện tại. Generator sau đó xem xét các đoạn truy xuất để tạo những phương án tiếp nối và đánh giá chúng. Ngược lại, nếu không cần truy xuất, mô hình tiếp tục sinh văn bản từ ngữ cảnh hiện tại. Vì quyết định được đưa ra trong quá trình tạo sinh, Self-RAG có thể truy xuất nhiều lần đối với một câu trả lời dài, hoặc hoàn toàn bỏ qua truy xuất khi tác vụ không yêu cầu chứng cứ bên ngoài [3].

Ngoài việc dự đoán trực tiếp token điều khiển, Self-RAG còn cho phép sử dụng một ngưỡng xác suất để chủ động điều chỉnh tần suất truy xuất. Gọi r^+ là lựa chọn *Retrieve=Yes*, $\mathcal{R}_{\text{Ret}}$ là tập các lựa chọn thuộc nhóm *Retrieve*, và τ là ngưỡng do hệ thống đặt ra. Một quyết định truy xuất có thể được biểu diễn như sau:

$$\text{Retrieve}_t = \begin{cases} 1, & \dfrac{p_\psi(r^+ \mid x, y_{<t})}{\sum_{r \in \mathcal{R}_{\text{Ret}}} p_\psi(r \mid x, y_{<t})} > \tau, \\ 0, & \text{ngược lại.} \end{cases} \tag{23}$$

Trong đó, tử số là xác suất của lựa chọn truy xuất, còn mẫu số chuẩn hóa theo các lựa chọn điều khiển có thể có. Việc hạ ngưỡng τ làm hệ thống dễ kích hoạt truy xuất hơn, thích hợp khi yêu cầu xác minh thông tin được ưu tiên. Ngược lại, tăng ngưỡng có thể giảm số lần gọi Retriever, phù hợp hơn với các tác vụ không phụ thuộc nhiều vào tri thức bên ngoài. Đây là cơ chế điều chỉnh tại thời điểm suy luận, không phải quá trình cập nhật trọng số của mô hình [3].

C. Hệ thống token phản biện (Reflection Tokens)

Reflection Tokens là các token đặc biệt được bổ sung vào tập từ vựng và được mô hình học cách dự đoán cùng với token văn bản thông thường. Theo Asai *et al.*, chúng gồm một nhóm điều khiển truy xuất (*Retrieve*) và ba nhóm đánh giá (*Critique*): *IsRel*, *IsSup* và *IsUse* [3]. Mỗi nhóm phản ánh một khía cạnh khác nhau của quá trình truy xuất hoặc chất lượng đầu ra.

a) Retrieve (Token điều khiển truy xuất): Nhóm *Retrieve* quyết định liệu mô hình có nên tìm thêm thông tin bên ngoài trước khi sinh phân đoạn tiếp theo hay không. Tín hiệu này được dự đoán dựa trên câu hỏi và phần câu trả lời đã có. Điểm đáng chú ý là quyết định truy xuất trở thành một phần của tiến trình tạo sinh thay vì một quy tắc được ấn định sẵn cho mọi truy vấn [3].

b) IsRel (Mức độ liên quan của tài liệu): Token *IsRel* đánh giá một đoạn văn bản d có chứa thông tin hữu ích đối với truy vấn x hay không, với hai mức *Relevant* và *Irrelevant*. Một tài liệu có điểm tương đồng ngữ nghĩa cao với câu hỏi vẫn có thể bị đánh giá là không hữu ích nếu thiếu dữ kiện cần thiết để giải quyết nhiệm vụ. Việc đánh giá này giúp mô hình hạn chế sử dụng những ngữ cảnh chỉ liên quan bề mặt [3].

c) IsSup (Mức độ được chứng cứ hỗ trợ): Token *IsSup* kiểm tra quan hệ giữa phân đoạn câu trả lời y_t và đoạn văn bản tham chiếu d, với ba mức *Fully supported*, *Partially supported* và *No support*. Khác với *IsRel*, vốn xét tài liệu có hữu ích cho truy vấn hay không, *IsSup* xét liệu các phát biểu cần kiểm chứng trong câu trả lời có được tài liệu xác nhận hay không. Sự phân biệt này có ý nghĩa khi tài liệu đúng chủ đề nhưng không cung cấp chứng cứ cho một khẳng định cụ thể của mô hình [3].

d) IsUse (Mức độ hữu ích tổng thể): Token *IsUse* đánh giá mức độ hữu ích của câu trả lời đối với yêu cầu đầu vào theo thang năm mức, từ 1 đến 5. Trong khi *IsRel* và *IsSup* tập trung vào tài liệu và chứng cứ, *IsUse* phản ánh chất lượng đáp ứng nhiệm vụ của phần văn bản được sinh. Nhờ sự kết hợp của ba nhóm đánh giá, Self-RAG có thể ưu tiên các câu trả lời vừa phù hợp với câu hỏi, vừa có căn cứ và có giá trị sử dụng [3].

Có thể hình dung một truy vấn yêu cầu xác định nguồn gốc một tên địa danh. Nếu mô hình kích hoạt *Retrieve*, nó sẽ tìm các đoạn tham chiếu, đánh giá đoạn nào liên quan bằng *IsRel*, sinh các phương án trả lời rồi sử dụng *IsSup* để kiểm tra mức độ được chứng cứ hỗ trợ. Sau đó, *IsUse* phản ánh mức độ hữu ích của nội dung đối với câu hỏi. Các nhãn này là tín hiệu do mô hình dự đoán, không phải phép xác minh đúng sai tuyệt đối; độ tin cậy của chúng phụ thuộc vào chất lượng huấn luyện và dữ liệu tham chiếu.

D. Quy trình huấn luyện và tùy biến suy luận

1) Huấn luyện Critic và Generator: Self-RAG sử dụng một Retriever bên ngoài, một mô hình đánh giá (*Critic*) và một mô hình sinh (*Generator*). Trong cách triển khai của Asai *et al.*, Critic và Generator được huấn luyện theo hai giai đoạn riêng; sau khi hoàn tất, Generator có thể tự sinh Reflection Tokens khi suy luận mà không cần gọi Critic như một mô hình đánh giá độc lập tại mỗi bước [3].

Trước hết, các tác giả tạo dữ liệu giám sát cho Critic bằng cách sử dụng một mô hình ngôn ngữ mạnh để gán nhãn phản biện cho những ví dụ câu hỏi, câu trả lời và tài liệu liên quan. Các nhãn này được dùng để tinh chỉnh Critic, giúp mô hình học cách dự đoán các Reflection Tokens tương ứng. Ký hiệu $\mathcal{D}_{\text{critic}}$ là tập dữ liệu giám sát của Critic gồm các ngữ cảnh c và nhãn phản biện r. Với c biểu diễn các thông tin điều kiện cần thiết cho từng nhóm token, mục tiêu học có thể viết gọn dưới dạng log-likelihood:

$$\max_{\phi} \; \mathbb{E}_{(c,r)\sim\mathcal{D}_{\text{critic}}} \log p_\phi(r \mid c). \tag{24}$$

Ở đây, ϕ biểu thị tham số của Critic; ngữ cảnh c có thể chứa truy vấn x, văn bản y và đoạn tài liệu d, tùy theo loại nhãn cần dự đoán. Đây là cách viết khái quát hóa mục tiêu học, không phải bản chép nguyên ký hiệu từ bài báo gốc [3].

Sau đó, Critic được dùng để bổ sung Reflection Tokens vào dữ liệu huấn luyện của Generator. Với những phân đoạn cần truy xuất, Retriever cung cấp các đoạn văn bản, Critic đánh giá tính liên quan và mức độ chứng cứ hỗ trợ, rồi các tín hiệu tương ứng được chèn xen kẽ vào văn bản. Tập dữ liệu mới

$\mathcal{D}_{\text{gen}}$ vì vậy chứa truy vấn x, chuỗi trả lời y, các nhãn phản biện r và ngữ cảnh tài liệu d (nếu có). Khi không kích hoạt truy xuất, ký hiệu d được hiểu là ngữ cảnh rỗng. Generator được huấn luyện bằng mục tiêu dự đoán token tiếp theo; biểu thức sau là dạng viết gọn khi gộp các vị trí có và không cần truy xuất:

$$\max_{\psi} \mathbb{E}_{(x,y,r,d)\sim\mathcal{D}_{\text{gen}}} \log p_\psi(y, r \mid x, d). \tag{25}$$

Trong quá trình tính Loss, phần văn bản tài liệu được truy xuất đóng vai trò ngữ cảnh điều kiện và không phải mục tiêu mà Generator cần học sao chép. Sau huấn luyện, Generator có thể đồng thời tạo văn bản và dự đoán token đánh giá dựa trên ngữ cảnh của chính quá trình sinh. Đây là điểm khác biệt với cách triển khai bắt buộc phải gọi một mô hình chấm điểm tách biệt mỗi khi cần kiểm tra đầu ra [3].

2) Tree-decoding và điều chỉnh tại thời điểm suy luận: Khi cần truy xuất, Self-RAG lấy K đoạn văn bản và tạo các phương án tiếp nối cho phân đoạn hiện tại dựa trên từng đoạn. Những ứng viên này được đánh giá bằng xác suất sinh nội dung và các Reflection Tokens. Asai *et al.* sử dụng tìm kiếm theo chùm ở cấp phân đoạn (*segment-level beam search*), có thể xem là một quá trình mở rộng các nhánh ứng viên để lựa chọn chuỗi đầu ra có điểm phù hợp nhất [3].

Cụ thể, với phân đoạn ứng viên y_t và tài liệu d, điểm đánh giá được mô tả như sau:

$$\begin{aligned} f(y_t, d) = p_\psi(y_t \mid x, d, y_{<t}) \\ + S_{\text{crit}}(y_t, d), \end{aligned} \tag{26}$$

trong đó điểm phản biện S_{crit} là tổng có trọng số của các nhóm token đánh giá $\mathcal{G} = \{\text{IsRel}, \text{IsSup}, \text{IsUse}\}$:

$$S_{\text{crit}}(y_t, d) = \sum_{G\in\mathcal{G}} w^G s_t^G(y_t, d). \tag{27}$$

Điểm $s_t^G(y_t, d)$ được tính từ xác suất chuẩn hóa của nhãn phản biện mong muốn $\hat{r}_G$ trong nhóm G:

$$s_t^G(y_t, d) = \frac{p_\psi(\hat{r}_G \mid c_t^G)}{\sum_{r\in\mathcal{R}_G} p_\psi(r \mid c_t^G)}, \tag{28}$$

trong đó $\mathcal{R}_G$ là tập các nhãn thuộc nhóm G, $\hat{r}_G$ là nhãn được ưu tiên và c_t^G biểu diễn ngữ cảnh khả dụng tại vị trí dự đoán nhóm token G (có thể khác nhau giữa các nhóm). Hệ số w^G điều chỉnh tầm quan trọng tương đối của từng tiêu chí. Đây là hàm chấm điểm phục vụ giải mã, không phải công thức xác suất biên duyên hóa của RAG-Sequence [3].

Ví dụ, nếu yêu cầu đặt trọng tâm vào khả năng kiểm chứng, hệ thống có thể tăng trọng số w^{IsSup} để ưu tiên những phân đoạn được tài liệu hỗ trợ. Với tác vụ có tính sáng tạo cao hơn, trọng số này có thể được giảm tương đối so với các tiêu chí khác. Mô hình cũng có thể áp dụng ràng buộc cứng (*hard constraints*), chẳng hạn loại bỏ các ứng viên bị đánh giá là *No support*, thay vì chỉ cộng điểm mềm (*soft constraints*). Ở mỗi bước, thuật toán giữ lại một số nhánh có điểm cao và tiếp tục mở rộng cho đến khi câu trả lời hoàn thành [3].

a) Ví dụ minh họa việc chấm điểm ứng viên: Giả sử trong một bước giải mã, hệ thống cần lựa chọn giữa hai phân đoạn y_A và y_B. Đặt các trọng số cho ba nhóm đánh giá lần lượt là $w^{\text{IsRel}} = 1$, $w^{\text{IsSup}} = 1$ và $w^{\text{IsUse}} = 0.5$. Với các bộ điểm giả định $\mathbf{s}_A = (0.90, 0.85, 0.80)$ và $\mathbf{s}_B = (0.65, 0.30, 0.90)$ (theo thứ tự IsRel, IsSup, IsUse), ta có:

$$\begin{aligned} S_{\text{crit}}(y_A, d_A) &= 0.90 + 0.85 + 0.5(0.80) = 2.15, \\ S_{\text{crit}}(y_B, d_B) &= 0.65 + 0.30 + 0.5(0.90) = 1.40. \end{aligned} \tag{29}$$

Mặc dù y_B nhận điểm cao hơn ở tiêu chí *IsUse*, y_A vẫn nhận điểm phản biện cao hơn nhờ mức liên quan và độ hỗ trợ từ chứng cứ tốt hơn. Nếu xác suất sinh của hai phân đoạn lần lượt là 0.15 và 0.20, hàm chấm điểm trong (26) cho kết quả $f(y_A, d_A) = 2.30$ và $f(y_B, d_B) = 1.60$. Các số liệu này chỉ minh họa cơ chế cộng điểm có trọng số; chúng không phải kết quả thực nghiệm của Self-RAG.

Khả năng thay đổi ngưỡng truy xuất τ và các trọng số w^G cho phép Self-RAG điều chỉnh hành vi của mô hình theo tác vụ tại thời điểm suy luận (*inference-time customization*). Việc này không yêu cầu huấn luyện lại Generator mỗi lần thay đổi tiêu chí ưu tiên. Tuy nhiên, cơ chế tự đánh giá không tự động bảo đảm mọi câu trả lời đều đúng; chất lượng cuối cùng vẫn phụ thuộc vào khả năng dự đoán Reflection Tokens, tài liệu được truy xuất và chiến lược giải mã [3].

Tóm lại, Self-RAG mở rộng kiến trúc RAG từ quy trình truy xuất và sinh văn bản sang một cơ chế có khả năng quyết định truy xuất và tự đánh giá ngay trong quá trình tạo sinh. Thông qua Reflection Tokens và giải mã ở cấp phân đoạn, mô hình có thể cân bằng giữa nhu cầu bổ sung tri thức, mức độ được chứng cứ hỗ trợ và tính hữu ích của câu trả lời mà không cần cố định một chiến lược truy xuất cho mọi truy vấn [3].

V. Ứng dụng downstream và khung đánh giá RAG

Sau khi trình bày các kiến trúc RAG ở những mục trước, mục này xem xét hai vấn đề thực tiễn: RAG được áp dụng và kiểm chứng trên những nhiệm vụ nào, và chất lượng của một hệ thống RAG được đo như thế nào. Do kết quả cuối cùng phụ thuộc đồng thời vào bộ truy xuất, kho tri thức và mô hình sinh, việc đánh giá không thể chỉ dựa trên đáp án cuối [2], [5].

A. Các miền nhiệm vụ thực nghiệm

1) Hỏi đáp ngắn và hỏi đáp miền mở: Trong hỏi đáp miền mở (*open-domain QA*), hệ thống phải trả lời câu hỏi mà không được cho trước tài liệu chứa đáp án; đáp án thường là một thực thể hoặc cụm từ ngắn. Đây cũng là nhóm tác vụ mà Lewis *et al.* sử dụng để đánh giá RAG nguyên bản [1]. Ba tập dữ liệu thường gặp là:

- **Natural Questions (NQ):** câu hỏi thật do người dùng nhập vào Google Search, có đáp án gán nhãn dựa trên Wikipedia.
- **TriviaQA:** khoảng 95 nghìn cặp hỏi–đáp do những người yêu thích đố vui biên soạn; câu hỏi thường dài và phức hợp.
- **PopQA:** khoảng 14 nghìn câu hỏi về thực thể, có ghi lại độ phổ biến của thực thể. Tập này làm nổi bật việc mô

hình thuần tham số trả lời kém với tri thức ít phổ biến (*long-tail*), vốn là vùng RAG có lợi thế nhất.

Độ đo chuẩn là khớp chính xác (*Exact Match*, EM) và F_1 ở mức token, sau khi chuẩn hóa chuỗi (chữ thường, bỏ dấu câu và mạo từ):

$$\mathrm{EM} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{a}_i \in A_i], \qquad F_1 = \frac{2PR}{P+R}, \tag{30}$$

trong đó $\hat{a}_i$ là đáp án dự đoán, A_i là tập đáp án chấp nhận được, còn P và R lần lượt là độ chính xác và độ phủ token. Vì mô hình sinh thường trả lời bằng câu đầy đủ, một số công trình thay EM nghiêm ngặt bằng độ chính xác dạng "chứa đáp án" [3].

Ví dụ minh họa. Giả sử đáp án chuẩn là "Seine" và mô hình trả lời "the Seine river". Sau chuẩn hóa, dự đoán còn hai token ("seine", "river"), nên $P = 1/2$, $R = 1$. Khi đó $\mathrm{EM} = 0$ vì hai chuỗi không trùng khớp hoàn toàn, nhưng $F_1 = \frac{2\times 0.5\times 1}{0.5+1} \approx 0.667$. Ví dụ cho thấy F_1 ghi nhận được mức đúng một phần mà EM bỏ qua.

2) Hỏi đáp dạng văn bản dài và sinh tiểu sử: Với hỏi đáp dạng dài (*long-form QA*), câu trả lời gồm nhiều câu, tổng hợp từ nhiều nguồn. ASQA xét câu hỏi mơ hồ có nhiều cách hiểu, đòi hỏi câu trả lời bao quát các cách hiểu đó; ELI5 gồm các câu hỏi mở dạng "giải thích đơn giản". Ở nhiệm vụ sinh tiểu sử (*biography generation*), mô hình viết tiểu sử một nhân vật và độ đúng thực tế được đo bằng **FActScore**: tách văn bản thành các sự kiện nguyên tử (*atomic facts*) rồi tính tỷ lệ sự kiện được nguồn tri thức hỗ trợ:

$$\mathrm{FActScore}(y) = \frac{1}{|\mathcal{A}_y|}\sum_{a\in\mathcal{A}_y} \mathbb{1}\big[a \text{ được nguồn hỗ trợ}\big]. \tag{31}$$

Chẳng hạn, một tiểu sử được tách thành 10 sự kiện nguyên tử, trong đó 8 sự kiện có căn cứ thì $\mathrm{FActScore} = 8/10 = 0.8$. Với nhóm nhiệm vụ này, ngoài FActScore còn thường dùng ROUGE và độ chính xác/độ phủ của trích dẫn.

3) Kiểm chứng thông tin: Trong kiểm chứng thông tin (*fact verification*), đầu vào là một tuyên bố và hệ thống gán nhãn (được hỗ trợ, bị bác bỏ hoặc không đủ thông tin) dựa trên chứng cứ truy xuất. **FEVER** gồm hơn 185 nghìn tuyên bố tạo từ các câu trong Wikipedia, kèm chứng cứ; **PubHealth** gồm các tuyên bố về sức khỏe cộng đồng, là miền chuyên ngành có rủi ro cao. Độ đo chính là độ chính xác nhãn; riêng FEVER còn có *FEVER score*, chỉ tính đúng khi nhãn đúng *và* chứng cứ được truy xuất đầy đủ, nên phản ánh đồng thời chất lượng truy xuất và suy luận. Bảng I tóm tắt ba nhóm nhiệm vụ.

B. Khung và chỉ số đánh giá RAG

1) Bộ ba chỉ số chất lượng chính: Để xác định lỗi nằm ở truy xuất hay ở bước sinh, các khung đánh giá hiện đại tách riêng ba mối quan hệ giữa câu hỏi q, ngữ cảnh truy xuất $c(q)$ và câu trả lời a. Bộ ba chỉ số này được RAGAS, ARES và TruLens cùng sử dụng, dù tên gọi có khác nhau đôi chút (TruLens gọi Answer Faithfulness là *Groundedness*) [5], [6].

Bảng I
CÁC MIỀN NHIỆM VỤ DOWNSTREAM CỦA RAG

Miền	Tập dữ liệu	Độ đo chính
QA ngắn, miền mở	NQ, TriviaQA, PopQA	EM, F_1
QA dài, tiểu sử	ASQA, ELI5, Biography	FActScore, ROUGE, citation P/R
Kiểm chứng	FEVER, PubHealth	Label accuracy, FEVER score

a) Context Relevance (độ liên quan ngữ cảnh): Đo mức độ $c(q)$ chỉ chứa thông tin cần thiết để trả lời q; chỉ số này phạt thông tin thừa. Trong RAGAS, LLM trích ra các câu then chốt trong $c(q)$ và điểm được tính bằng tỷ lệ câu được trích [5]:

$$\mathrm{CR} = \frac{\text{số câu then chốt được trích từ } c(q)}{\text{tổng số câu trong } c(q)}. \tag{32}$$

Điểm thấp cho thấy bộ truy xuất đưa vào nhiều đoạn nhiễu, gợi ý điều chỉnh chunking, mô hình embedding hoặc số đoạn K.

b) Answer Faithfulness (độ trung thực câu trả lời): Đo mức độ câu trả lời được suy ra từ ngữ cảnh truy xuất, qua đó phát hiện ảo giác. LLM tách câu trả lời thành tập phát biểu S rồi kiểm tra từng phát biểu có được $c(q)$ hỗ trợ hay không [5]:

$$F = \frac{|V|}{|S|}, \qquad V \subseteq S \text{ là các phát biểu được ngữ cảnh hỗ trợ.} \tag{33}$$

c) Answer Relevance (độ liên quan câu trả lời): Đo mức độ câu trả lời giải quyết trực tiếp câu hỏi, chưa xét tính đúng sự thật. LLM sinh ngược n câu hỏi q_i từ câu trả lời, rồi lấy trung bình độ tương đồng embedding với câu hỏi gốc [5]:

$$\mathrm{AR} = \frac{1}{n}\sum_{i=1}^{n} \mathrm{sim}(q, q_i). \tag{34}$$

Ví dụ minh họa. Xét một lượt hỏi đáp có các số liệu giả định như sau (không phải kết quả thực nghiệm):

1) *Context Relevance:* ngữ cảnh có 10 câu, trong đó 2 câu được trích là cần thiết, nên $\mathrm{CR} = 2/10 = 0.2$; bộ truy xuất đang đưa vào nhiều nội dung thừa.
2) *Faithfulness:* câu trả lời tách được 4 phát biểu, trong đó 3 phát biểu được ngữ cảnh hỗ trợ, nên $F = 3/4 = 0.75$; phát biểu còn lại là nội dung mô hình tự thêm vào.
3) *Answer Relevance:* từ câu trả lời sinh ra $n = 3$ câu hỏi với độ tương đồng 0.92, 0.88 và 0.80 so với câu hỏi gốc, nên $\mathrm{AR} = (0.92 + 0.88 + 0.80)/3 \approx 0.867$.

Ba điểm số khác nhau cho thấy các chỉ số tương đối độc lập: câu trả lời có thể rất liên quan đến câu hỏi nhưng vẫn chứa phát biểu thiếu căn cứ.

d) Liên hệ với Self-RAG: Các token phản biện của Self-RAG ở Mục IV có thể xem là phiên bản được huấn luyện sẵn trong mô hình của cùng ý tưởng này: *IsRel* tương ứng gần đúng với Context Relevance, *IsSup* với Answer Faithfulness, còn *IsUse* đo mức hữu ích tổng thể của câu trả lời [3]. Điểm

khác biệt là Self-RAG dùng các tín hiệu đó để điều khiển giải mã ngay trong lúc sinh, trong khi RAGAS hay TruLens đánh giá hệ thống từ bên ngoài sau khi đã sinh.

2) Các công cụ và benchmark nổi bật:

- **RAGAS:** khung đánh giá không cần tham chiếu (*reference-free*), dùng LLM để chấm Faithfulness, Answer Relevance và Context Relevance. Ưu điểm là triển khai nhanh, không cần nhãn người [5].
- **ARES:** báo cáo cùng ba điểm số nhưng huấn luyện giám khảo nhẹ cho từng chỉ số bằng dữ liệu tổng hợp, rồi dùng một tập nhỏ có nhãn người để hiệu chỉnh và ước lượng khoảng tin cậy [6].
- **TruLens:** thư viện mã nguồn mở đề xuất "RAG Triad" (Context Relevance, Groundedness, Answer Relevance), cho phép gắn vào ứng dụng để ghi lại và theo dõi từng lượt gọi.
- **RGB:** benchmark song ngữ Anh–Trung đo bốn năng lực của LLM khi dùng RAG: chịu nhiễu (*noise robustness*), từ chối khi thiếu thông tin (*negative rejection*), tích hợp thông tin (*information integration*) và chịu thông tin sai (*counterfactual robustness*) [7].
- **CRUD-RAG:** benchmark tiếng Trung phân loại ứng dụng RAG theo bốn thao tác Create, Read, Update, Delete, tương ứng các tác vụ tiếp nối văn bản, hỏi đáp, chỉnh sửa ảo giác và tóm tắt đa tài liệu; ngoài LLM, benchmark còn khảo sát ảnh hưởng của kích thước chunk, mô hình embedding và thuật toán truy xuất [8].

Có thể phân biệt hai nhóm: RAGAS, ARES, TruLens là công cụ chấm điểm có thể áp lên dữ liệu của chính người dùng, còn RGB và CRUD-RAG là bộ dữ liệu chuẩn để so sánh các hệ thống với nhau.

3) Hạn chế khi đánh giá: Các chỉ số dựa trên LLM làm giám khảo nhạy với mô hình và prompt được chọn, đồng thời có những thiên lệch đã biết; vì vậy nên hiệu chỉnh bằng một tập nhỏ có nhãn người, như ARES [6]. Dùng LLM để chấm một hệ thống cũng dựa trên LLM có thể che giấu lỗi chung. Bộ ba chỉ số không bao phủ độ trễ, chi phí hay tuân thủ chính sách, nên cần bổ sung chỉ số vận hành. Cuối cùng, benchmark cũ có nguy cơ đã nằm trong dữ liệu huấn luyện của LLM, và với miền rủi ro cao như y tế, điểm số tự động không thay thế được đánh giá của chuyên gia.

TÀI LIỆU

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, ver. 5, Mar. 2024.

[3] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. EACL (System Demonstrations)*, 2024, arXiv:2309.15217.

[6] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in *Proc. NAACL-HLT*, 2024, arXiv:2311.09476.

[7] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proc. AAAI*, vol. 38, no. 16, 2024, pp. 17754–17762.

[8] Y. Lyu *et al.*, "CRUD-RAG: A comprehensive Chinese benchmark for retrieval-augmented generation of large language models," arXiv:2401.17043, 2024.